

Comprendre le fonctionnement d'un site web

1- Architecture web



- Pour fonctionner le client et le serveur ont besoin d'une intelligence, un système d'exploitation

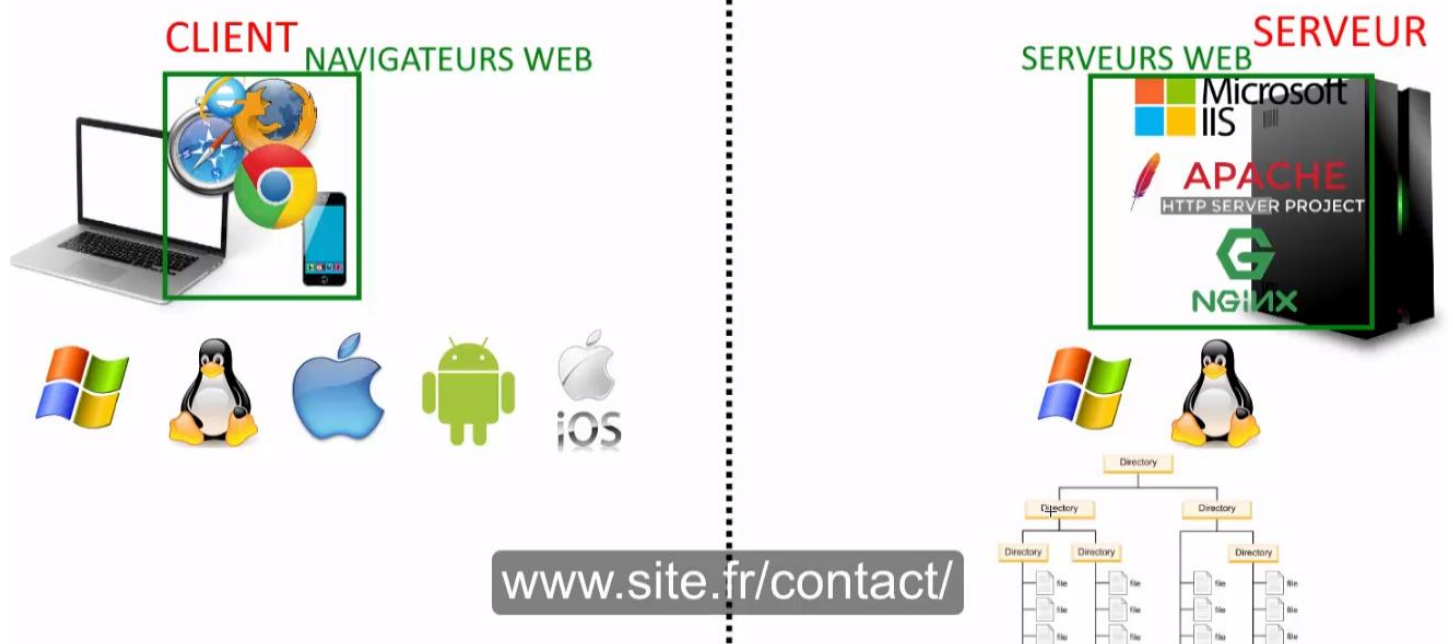


A savoir que coté intelligence serveur, la part de marché est divisée en deux entre Microsoft (windows server, AD, SQL server) et Linux (Ubuntu, CentoAs, FreeBSD).

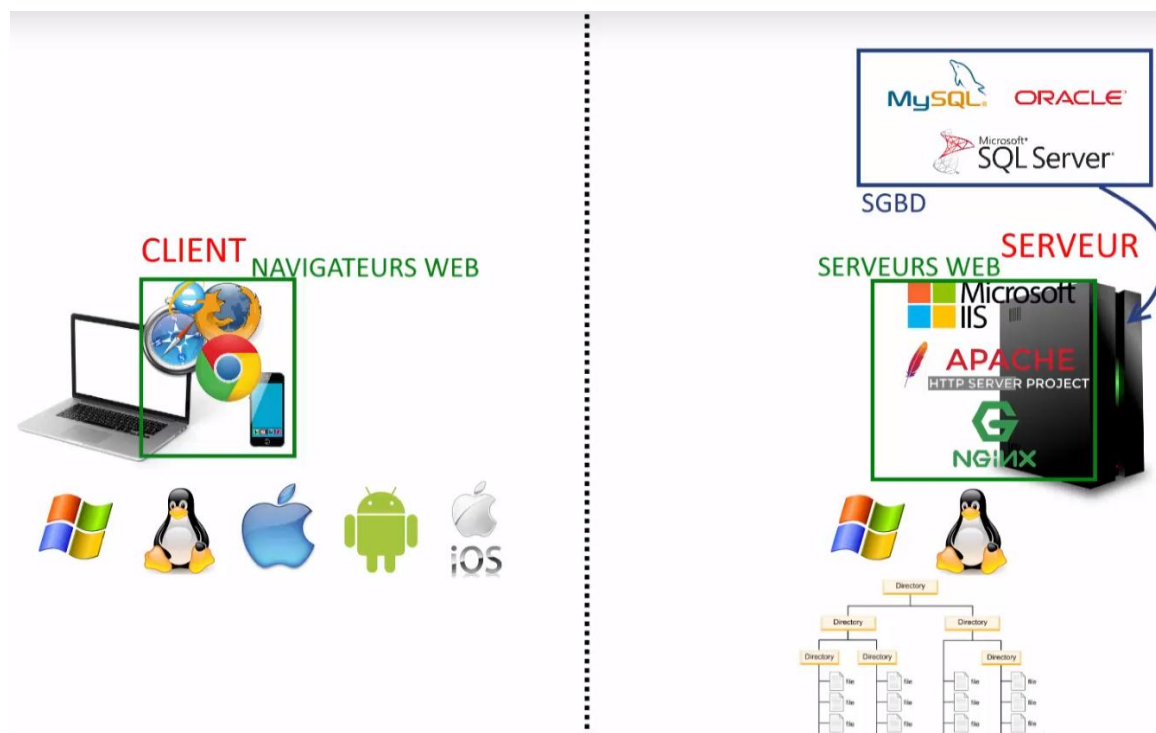
Le client, depuis son périphérique, utilise un navigateur web pour naviguer sur les sites web, du côté serveur on installera à partir du système d'exploitation ce qu'on appelle un serveur web



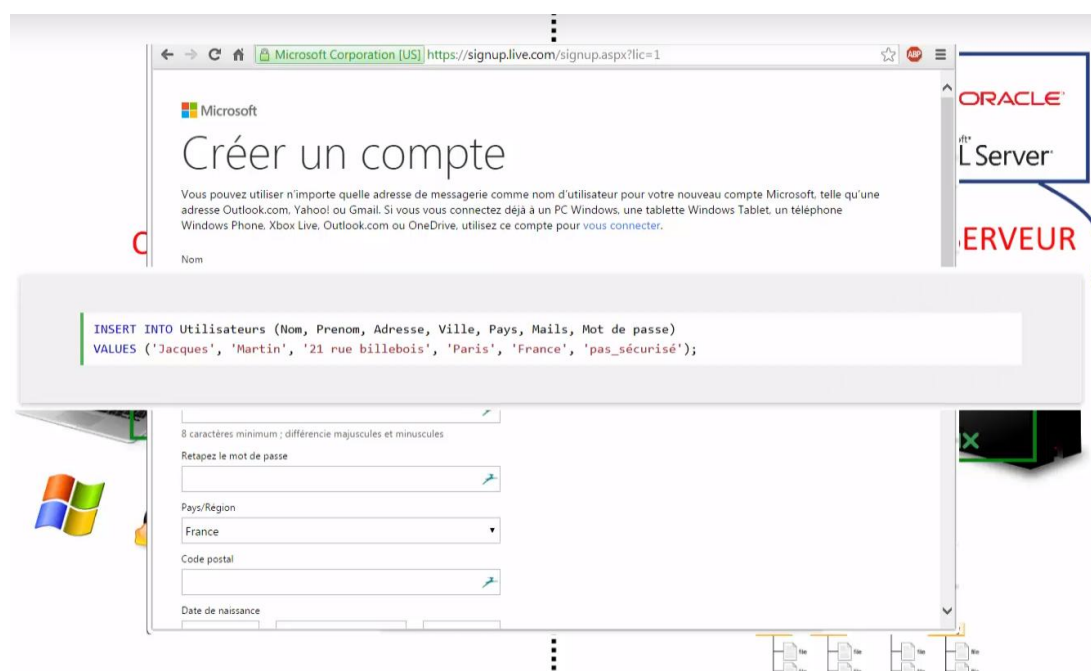
Un site web, c'est un ensemble de dossiers, de fichiers html, css, javascript etc... Ces fichiers sont disposés coté serveur en forme arborescence (comme un PC sans interface graphique). Ces fichiers sont ceux qui seront visités



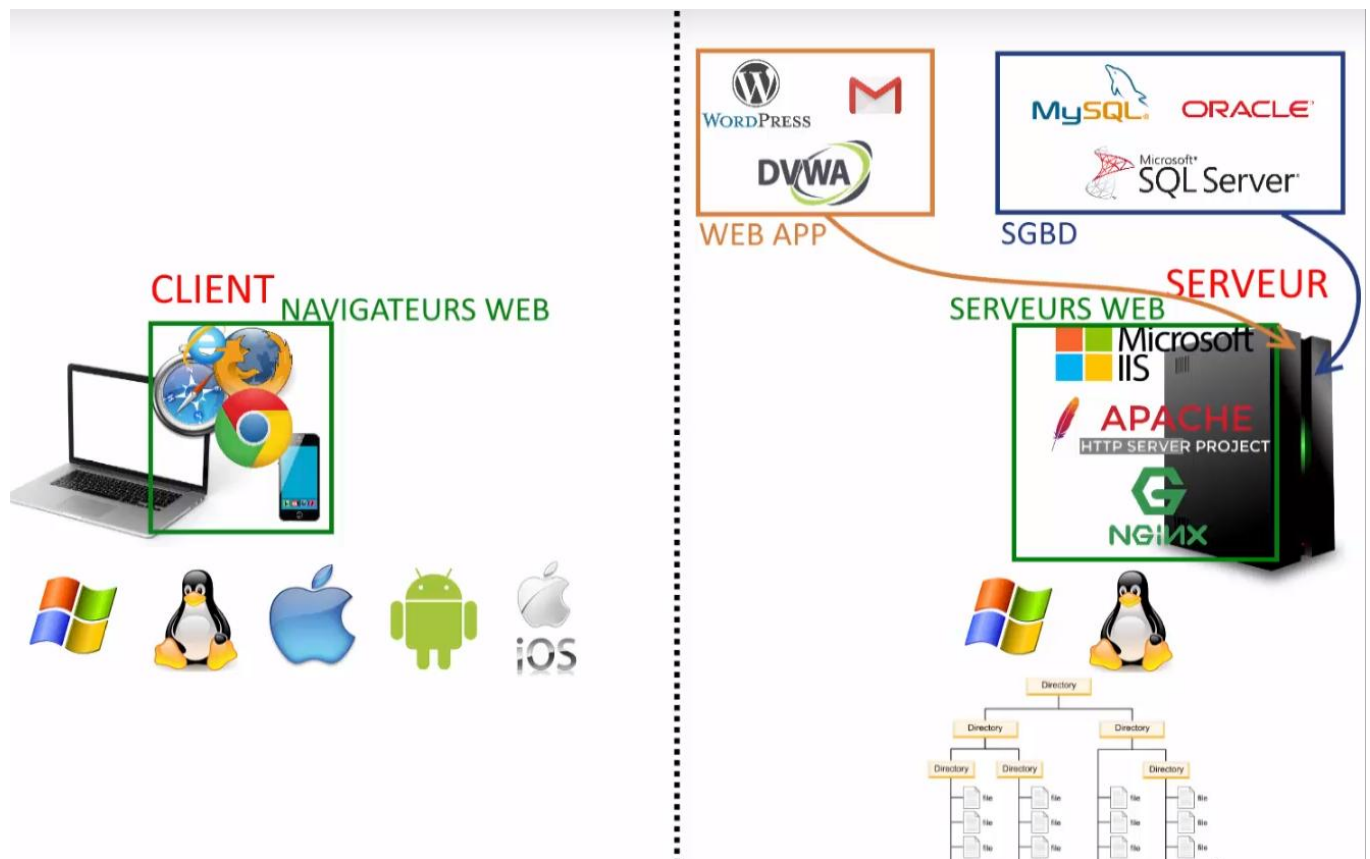
Coté serveur également (cerveau de l'application) pour que tout fonctionne il va falloir configurer pas mal d'outils comme les SGBD (système de gestion des bases de données) qui permettra de stocker des informations



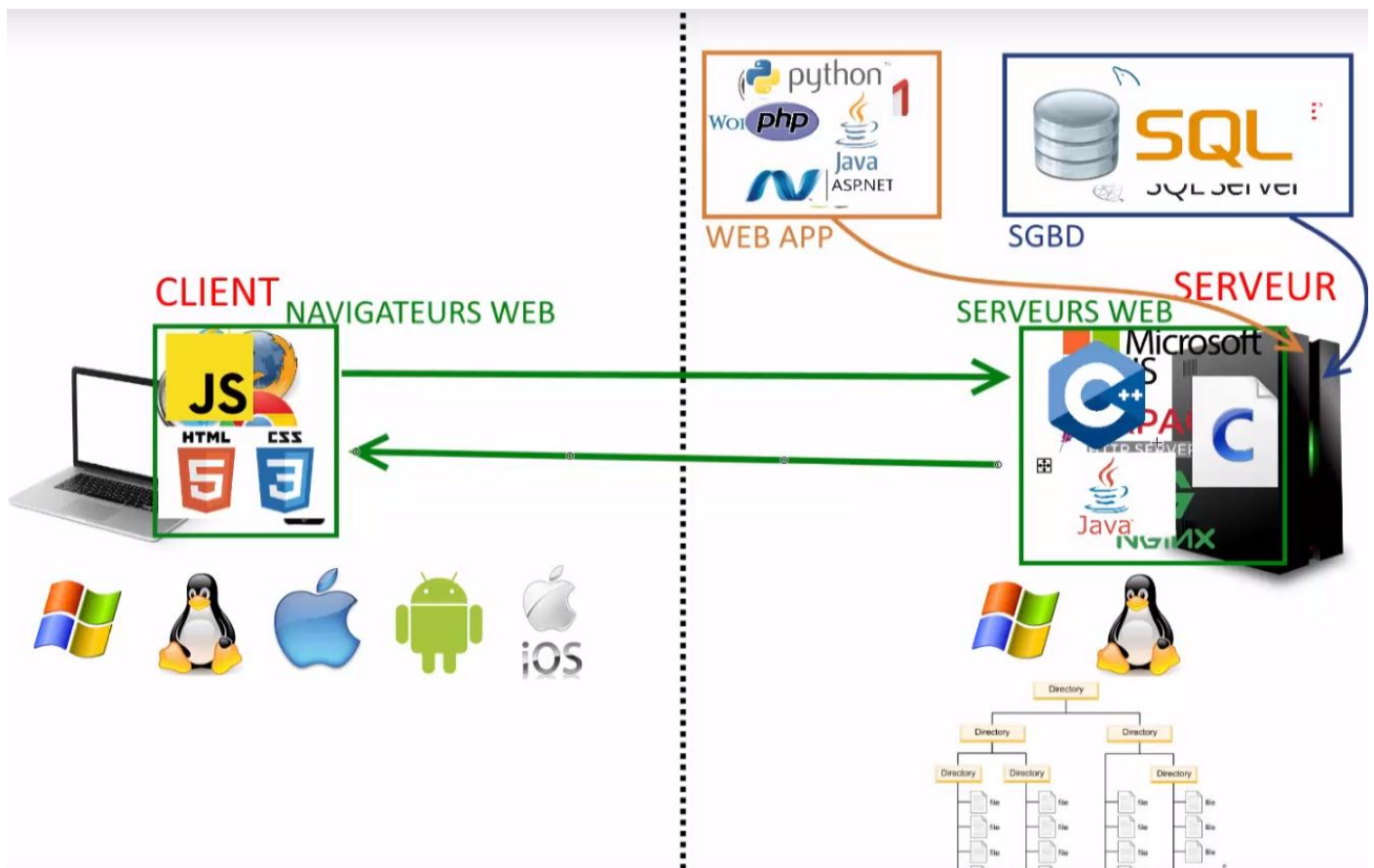
Cela permet par exemple de créer des comptes utilisateur



Sur ces serveurs, nous avons aussi des application web, qui sont des services en ligne qui permettent de faire une tâche bien précise à partir d'un interface web



Et pour finir, toutes ses technologies citées, utilise des langages de programmation pour communiquer



2- DNS (Domain Name System)

Afin de pouvoir nous orienter sur le web, les adresses IP sont converties en chaînes de caractères compréhensibles pour l'homme.

Cependant, les outils et technologies vue plus hauts, ont besoin d'adresses IP tel-qu'elles pour fonctionner, c'est là que le DNS intervient et va transformer le nom de domaine en adresse IP compréhensible pour les machines

Mais comment cela fonctionne ?

Le point de départ est le navigateur car c'est dans sa barre de recherche que sera renseignée l'adresse | domaine

1- Rechercher dans le cache

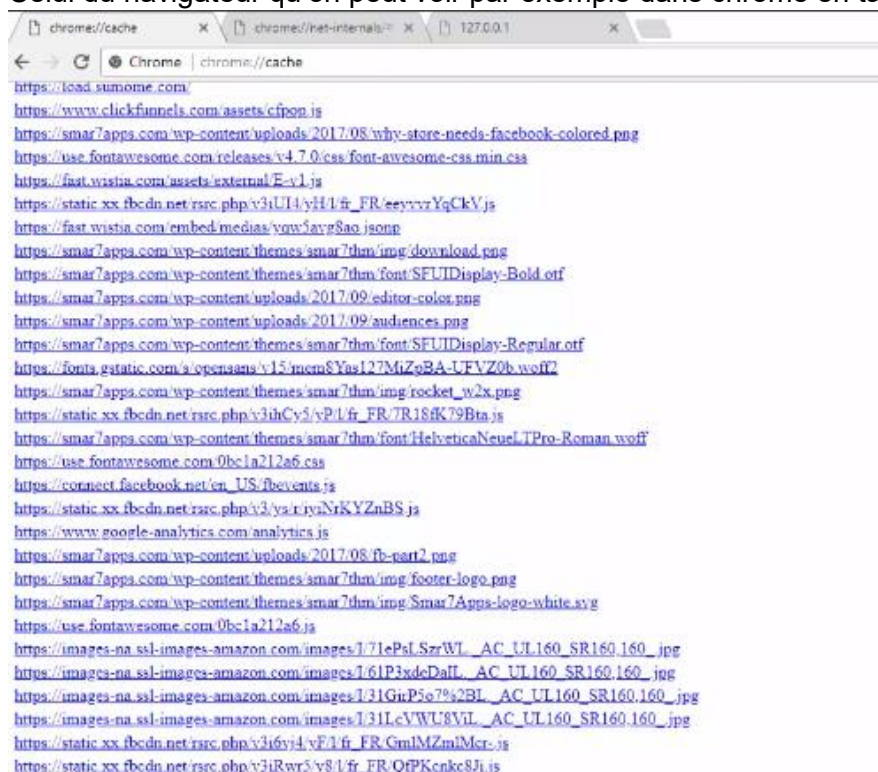
La première chose que fait le navigateur est de rechercher l'adresse en question dans son cache afin de vérifier s'il ne la connaît pas déjà.

Recherche
dans le cache



Il existe deux types de caches :

- Celui du navigateur qu'on peut voir par exemple dans chrome en tapant > *chrome://cache*



Il s'agit là de fichiers dits statiques car ils ne changent pas beaucoup (logos, fichiers images ou .woff pour les polices d'écritures). Ainsi, si le site a déjà été visité, nous pouvons récupérer ses données via le cache

- Le cache DNS (chrome//net-internals/#dns), qui lui enregistre un nom d'hôte avec sa correspondance d'adresse IP. A la différence du nom de domaine qui lui est générique, le nom d'hôte lui pointe nécessairement vers une adresse IP

Capturing halted

Appuyez sur **Echap** pour quitter le mode live

Import
Proxy
Events
Timeline
DNS
Sockets
Alt-Svc
HTTP/2
QUIC
Cache
Modules
Bandwidth
Prerender

• [View pending lookups](#)

Async DNS Configuration

- Internal DNS client enabled: false

Host resolver cache

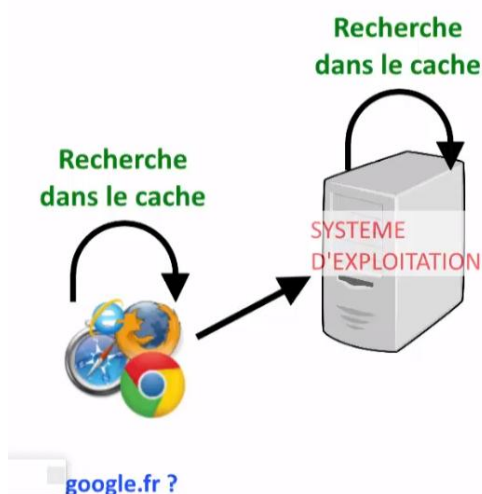
- Capacity: 1000

Current State

- Active entries: 8
- Expired entries: 277
- Network changes: 1

Hostname	Family	Addresses	TTL	Expires	Network changes
0-edge-chat.facebook.com	IPv4	31.13.90.2	-1000	2018-02-22 16:28:05.737 [Expired]	1
0.gravatar.com	IPv4	192.0.73.2	-1000	2018-02-22 16:27:52.240 [Expired]	1
1-edge-chat.facebook.com	IPv4	31.13.90.2	-1000	2018-02-22 16:28:15.488 [Expired]	1
1.gravatar.com	IPv4	192.0.73.2	-1000	2018-02-22 16:27:52.294 [Expired]	1
2-edge-chat.facebook.com	IPv4	31.13.90.2	-1000	2018-02-22 16:28:12.334 [Expired]	1
2.gravatar.com	IPv4	192.0.73.2	-1000	2018-02-22 16:27:52.299 [Expired]	1
4-edge-chat.facebook.com	IPv4	31.13.90.2	-1000	2018-02-22 16:28:06.421 [Expired]	1
4aefe439b171479ed763faf91395c42f.nrb.footprintdns.com	IPv4	13.107.6.254	-1000	2018-02-22 16:33:00.468 [Expired]	1
5-edge-chat.facebook.com	IPv4	31.13.90.2	-1000	2018-02-22 16:28:05.735 [Expired]	1
6-edge-chat.facebook.com	IPv4	31.13.90.2	-1000	2018-02-22 16:28:05.736 [Expired]	1
8-trouter-neu-a.drip.trouter.io	IPv4	13.79.147.44	-1000	2018-02-22 16:28:29.787 [Expired]	1
a.config.skype.com	IPv4	23.101.158.111	-1000	2018-02-22 16:28:22.306 [Expired]	1
a.liv.skype.com	IPv4	13.77.92.139	-1000	2018-02-22 16:27:49.884 [Expired]	1
a.optnmstr.com	IPv4	23.111.11.83	-1000	2018-02-22 16:28:38.795 [Expired]	1
a.quora.com	IPv4	192.229.221.122	-1000	2018-02-22 16:28:15.955 [Expired]	1
aax-eu.amazon-adsystem.com	IPv4	52.94.220.16	-1000	2018-02-22 16:28:12.755 [Expired]	1
accounts.google.com	IPv4	172.217.17.45	-1000	2018-02-22 16:29:16.235 [Expired]	1
accounts.youtube.com	IPv4	172.217.17.46	-1000	2018-02-22 16:28:10.407 [Expired]	1
ad.admt.com	IPv4	31.13.90.9	-1000	2018-02-22 16:28:17.574 [Expired]	1
ad.doubleclick.net	IPv4	172.217.17.134	-1000	2018-02-22 16:27:48.974 [Expired]	1
ads.scorecardresearch.com	IPv4	2.19.43.224	-1000	2018-02-22 16:28:06.382 [Expired]	1
ads.yahoo.com	IPv4	217.12.15.54 217.12.15.83	-1000	2018-02-22 16:28:18.640 [Expired]	1
adservice.google.com	IPv4	216.58.212.130	-1000	2018-02-22 16:27:52.237 [Expired]	1
adservice.google.fr	IPv4	172.217.20.98	-1000	2018-02-22 16:39:13.377	1
		216.58.212.234 216.58.211.106 216.58.212.202 172.217.20.74			
asia-pacific.google.com	IPv4	173.217.10.303	-1000	2018-02-22 16:38:35.305 [Expired]	1

Si le navigateur vient de découvrir l'adresse, donc rien dans son cache et aucune IP connu, il se dirige alors vers le système d'exploitation, qui lui aussi possède son propre cache qu'on peut observer sur windows par exemple en tapant dans le cmd > *ipconfig /displaydns*



```

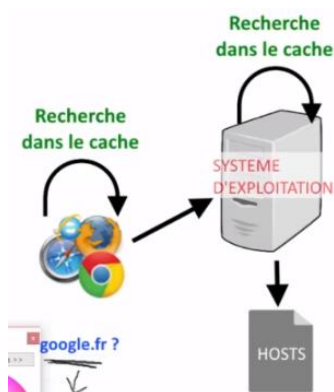
Invite de commandes

Nom d'enregistrement : ns47.domaincontrol.com
Type d'enregistrement : 28
Durée de vie : 14131
Longueur de données : 16
Section : Supplémentaire
Enregistrement AAAA : 2607:f208:206::18

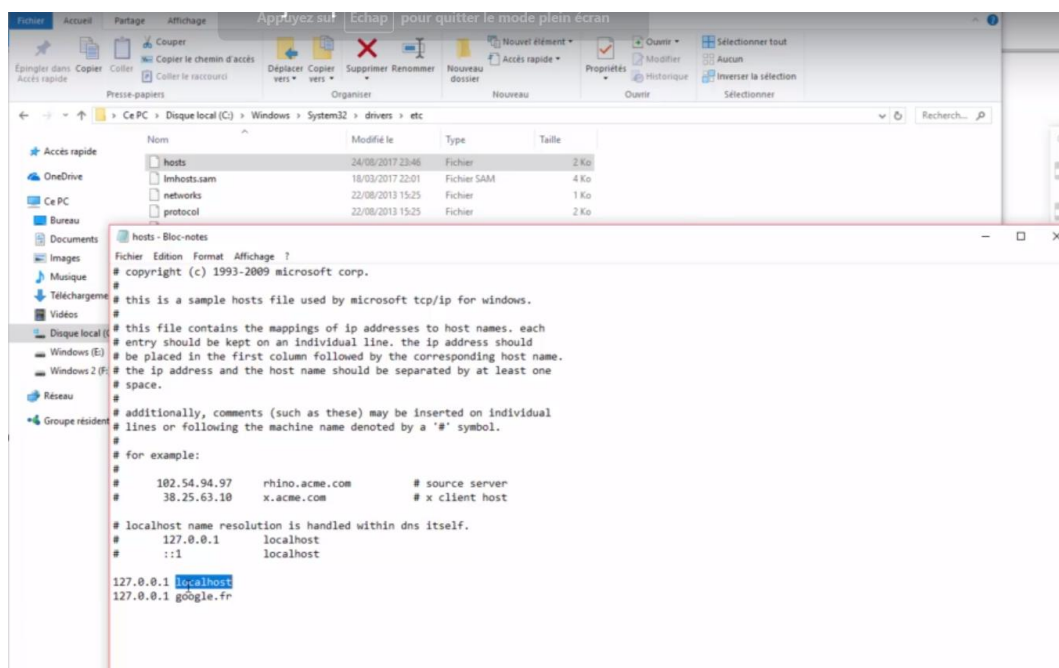
Nom d'enregistrement : ns48.domaincontrol.com
Type d'enregistrement : 1
Durée de vie : 14131
Longueur de données : 4
Section : Supplémentaire
Enregistrement (hôte) : 208.109.254.24

Nom d'enregistrement : ns48.domaincontrol.com
Type d'enregistrement : 28
Durée de vie : 14131
Longueur de données : 16
Section : Supplémentaire
Enregistrement AAAA : 2607:f208:302::18
  
```

Il est également possible de créer un cache manuel sur votre système via le fichier host

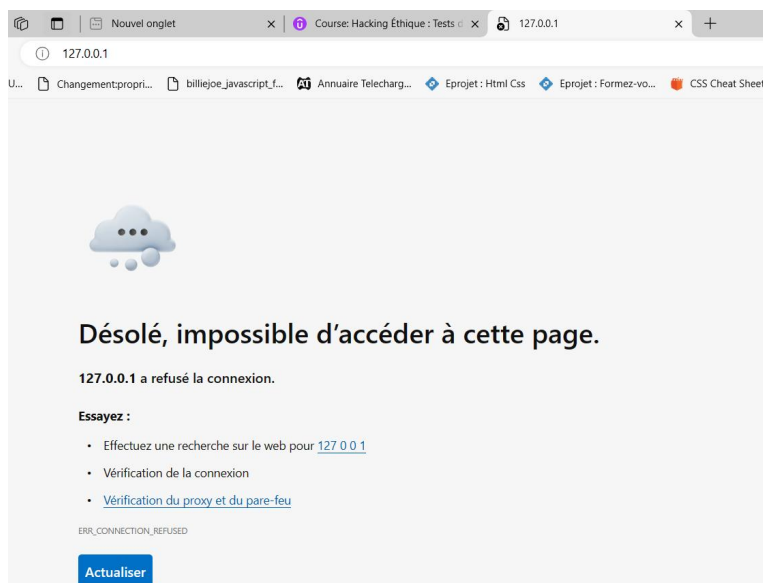


Accessible depuis le chemin > disque (C :)/Windows/Système32/drivers/etc



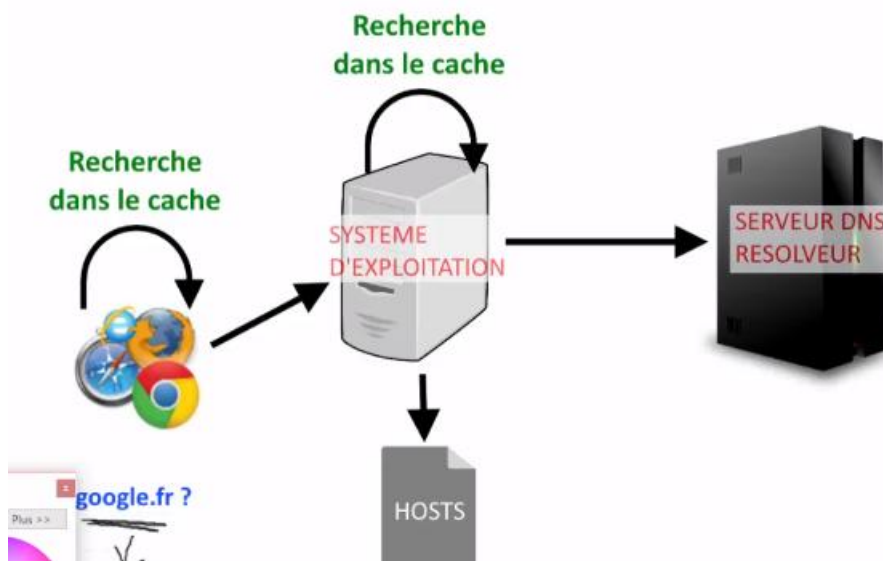
Par défaut nous y avons notre localhost. Nous pouvons y placer une adresse IP et un nom de domaine.

L'adresse IP 127.0.0.1 est une adresse non visible sur le net mais uniquement exploitable depuis un serveur web installé en local, ainsi donc si vous n'avez pas configuré l'installation, le navigateur répondra par



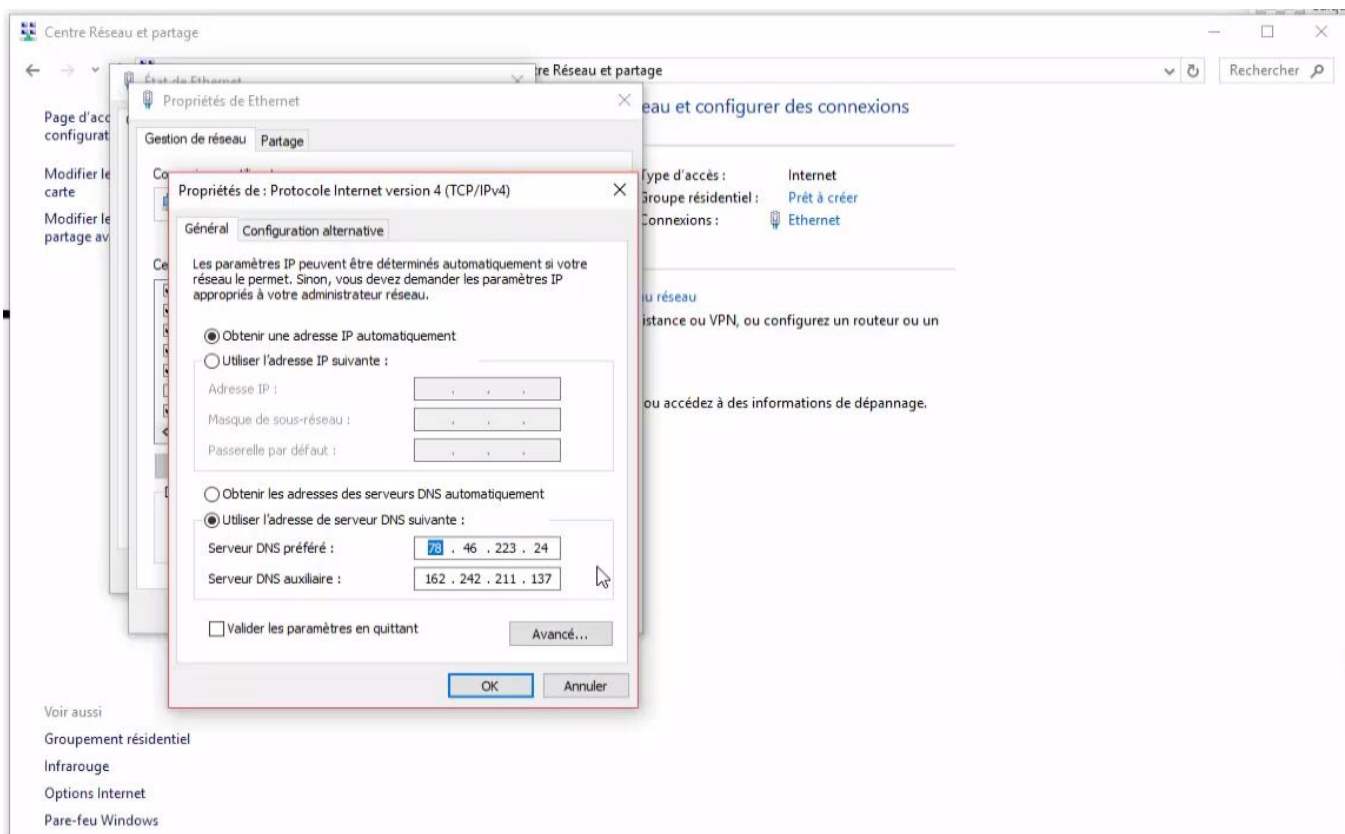
Ainsi donc, il nous est possible de bloquer un site web. Dans notre exemple, nous avons lié google.fr à l'IP 127.0.0.1 (inexploitable sans serveur web localement configuré). Par conséquent à chaque fois que cette adresse sera recherchée dans le navigateur l'erreur « impossible d'accéder à cette page » apparaîtra

Continuons, notre process, jusqu'à présent le navigateur n'a toujours pas trouver l'adresse correspondant à nom qui lui a été soumis, il va donc se diriger désormais vers le serveur DNS **résolveur** .



Ce serveur est configuré dans les paramètres réseaux de notre système d'exploitation.

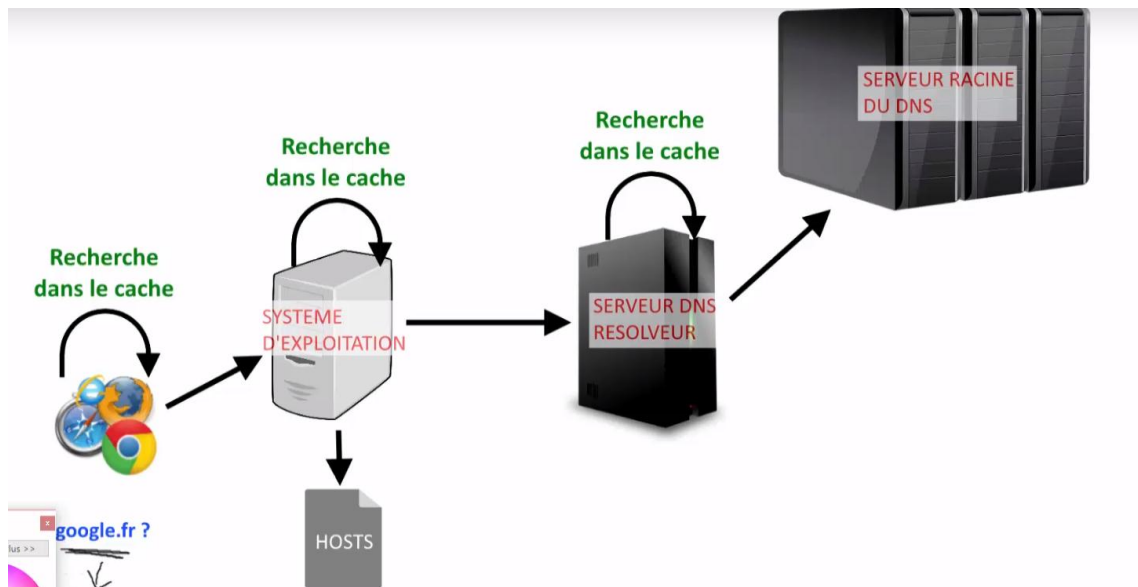
- Panneau de configuration > Centre Réseau et partage > Etat de Ethernet > Propriétés de Ethernet > Gestion de réseau > Propriété



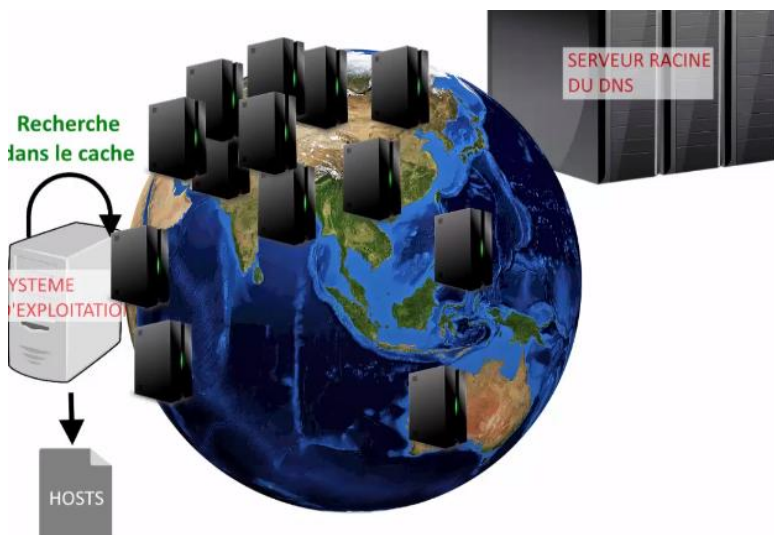
Il va donc prendre le relais, et lui aussi interroger son propre cache qui contient une énorme liste de correspondance de tous les clients du fournisseur d'accès à internet.

Supposons que nous sommes vraiment malchanceux et que le resolveur non plus ne connaît cette adresse !

Le processus va donc continuer, le résolveur va se retourner vers le serveur racine du DNS

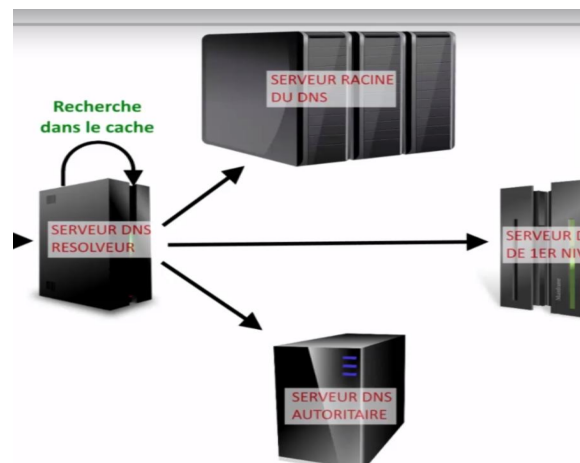
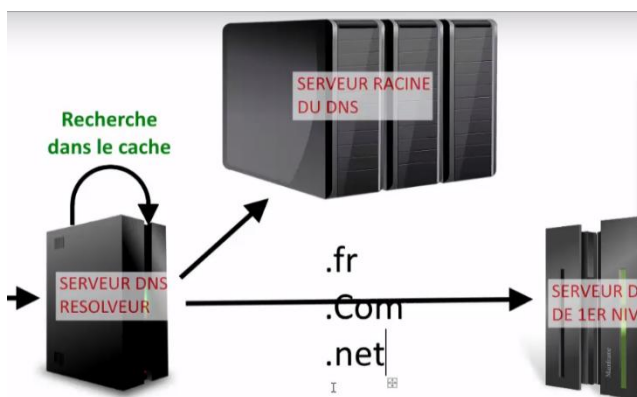


Il existe présentement, 13 serveurs racine du DNS répartis dans le monde et 12 organisations différentes également réparties dans le monde

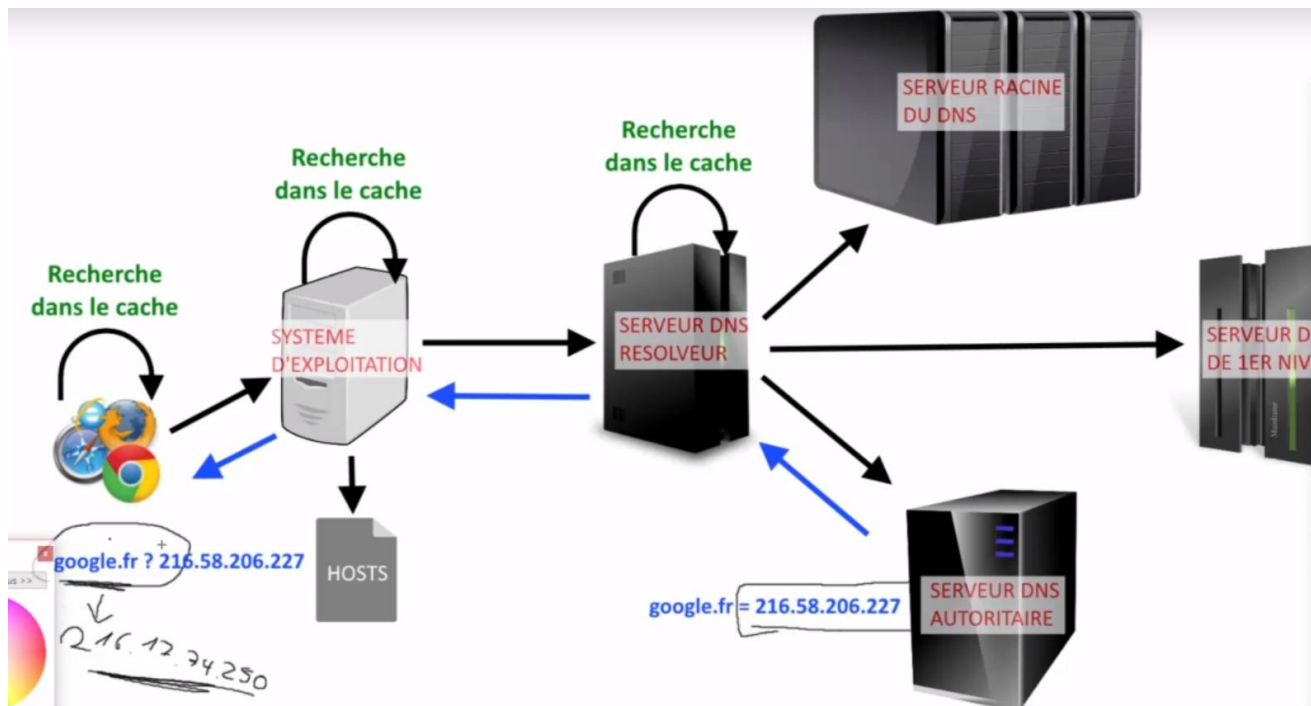


Les opérateurs vont donc placer plusieurs machines physiques regroupées derrière chaque nom d'hôte afin d'assurer une redondance et une protection en cas de panne.

Le rôle de ses serveurs racine dns est de nous rediriger ensuite vers un serveur dns de premier niveau qui lui nous redirige vers un serveur dns autoritaire qui nous donnera la bonne information



Serveur dns de premier niveau car le premier niveau du domaine dans notre cas google.fr est .fr et le sous-domaine google. Ensuite, le serveur dns autoritaire lui, à partir de tous ses éléments, saura nous donner la solution. Cette réponse sera donc transmise aux autres machines du processus qui s'empresseront de la conserver dans leur cache



Voici un schéma numéroté simplifié illustrant le processus de résolution DNS :

1. L'utilisateur envoie une requête pour accéder à un site web.
2. Le navigateur envoie une requête DNS au serveur DNS local.
3. Le serveur DNS local vérifie s'il dispose de l'adresse IP correspondante dans son cache. Si oui, il la renvoie au navigateur. Sinon, il poursuit le processus.
4. Le serveur DNS local envoie une requête DNS au serveur DNS racine.
5. Le serveur DNS racine renvoie une liste de serveurs DNS de premier niveau correspondant au domaine demandé.
6. Le serveur DNS local envoie une requête DNS au serveur DNS de premier niveau.
7. Le serveur DNS de premier niveau renvoie l'adresse IP du serveur DNS autoritaire pour le domaine demandé.
8. Le serveur DNS local envoie une requête DNS au serveur DNS autoritaire.
9. Le serveur DNS autoritaire renvoie l'adresse IP correspondant au domaine demandé.
10. Le serveur DNS local met en cache l'adresse IP pour les futures requêtes et la renvoie au navigateur.
11. Le navigateur utilise l'adresse IP pour établir une connexion avec le serveur web du site demandé.

Cas d'exemple avec DIG

La commande "dig" est un outil de ligne de commande utilisé dans les systèmes Linux pour effectuer des requêtes DNS (Domain Name System). Le DNS est un système qui associe les noms de domaine (comme `www.example.com`) aux adresses IP correspondantes.

La commande "dig" permet d'interroger les serveurs DNS pour obtenir des informations sur les enregistrements DNS d'un domaine spécifique. Voici quelques utilisations courantes de la commande "dig" :

1. Résolution de nom : Vous pouvez utiliser "dig" pour résoudre un nom de domaine en une adresse IP. Par exemple, en exécutant la commande `"dig example.com"`, vous obtiendrez l'adresse IP associée à ce domaine.
2. Requêtes spécifiques : "dig" permet d'effectuer différents types de requêtes DNS, tels que les requêtes de type A (adresse IP), les requêtes de type MX (serveur de messagerie), les requêtes de type NS (serveur de noms), etc. Vous pouvez spécifier le type de requête avec l'option "-t". Par exemple, `"dig -t MX example.com"` vous donnera les enregistrements MX associés à ce domaine.
3. Interrogation d'un serveur DNS spécifique : Par défaut, "dig" interroge le serveur DNS configuré sur votre système. Cependant, vous pouvez spécifier un serveur DNS spécifique à interroger en utilisant l'option "@". Par exemple, `"dig @8.8.8.8 example.com"` interrogera le serveur DNS de Google (8.8.8.8) pour obtenir les enregistrements DNS du domaine `example.com`.

La commande "dig" est un outil puissant pour effectuer des requêtes DNS et obtenir des informations détaillées sur les enregistrements DNS d'un domaine. Elle est couramment utilisée par les administrateurs système, les développeurs web et les utilisateurs qui souhaitent diagnostiquer les problèmes de résolution DNS ou obtenir des informations sur la configuration DNS d'un domaine spécifique.

➤ `dig @a.root-servers.net SOA fr`

```
File Actions Edit View Help
$ dig @a.root-servers.net SOA fr

; <<>> DiG 9.19.19-1-Debian <<>> @a.root-servers.net SOA fr
; (2 servers found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 24603
;; flags: qr rd; QUERY: 1, ANSWER: 0, AUTHORITY: 4, ADDITIONAL: 9
;; WARNING: recursion requested but not available

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;fr.                                IN      SOA

;; AUTHORITY SECTION:
fr.      172800 IN      NS      f.ext.nic.fr.
fr.      172800 IN      NS      d.nic.fr.
fr.      172800 IN      NS      g.ext.nic.fr.
fr.      172800 IN      NS      e.ext.nic.fr.

;; ADDITIONAL SECTION:
f.ext.nic.fr. 172800 IN      A      194.146.106.46
f.ext.nic.fr. 172800 IN      AAAA   2001:67c:1010:11::53
d.nic.fr.     172800 IN      A      194.0.9.1
d.nic.fr.     172800 IN      AAAA   2001:678:c::1
g.ext.nic.fr. 172800 IN      A      194.0.36.1
```

Liste des différents DNS autoritaires pour le domaine .fr

Nous allons donc demander à un de ces DNS de nous afficher les serveurs de noms autoritaires pour le domaine `google.fr`

➤ `dig @d.nic.fr SOA google.fr`

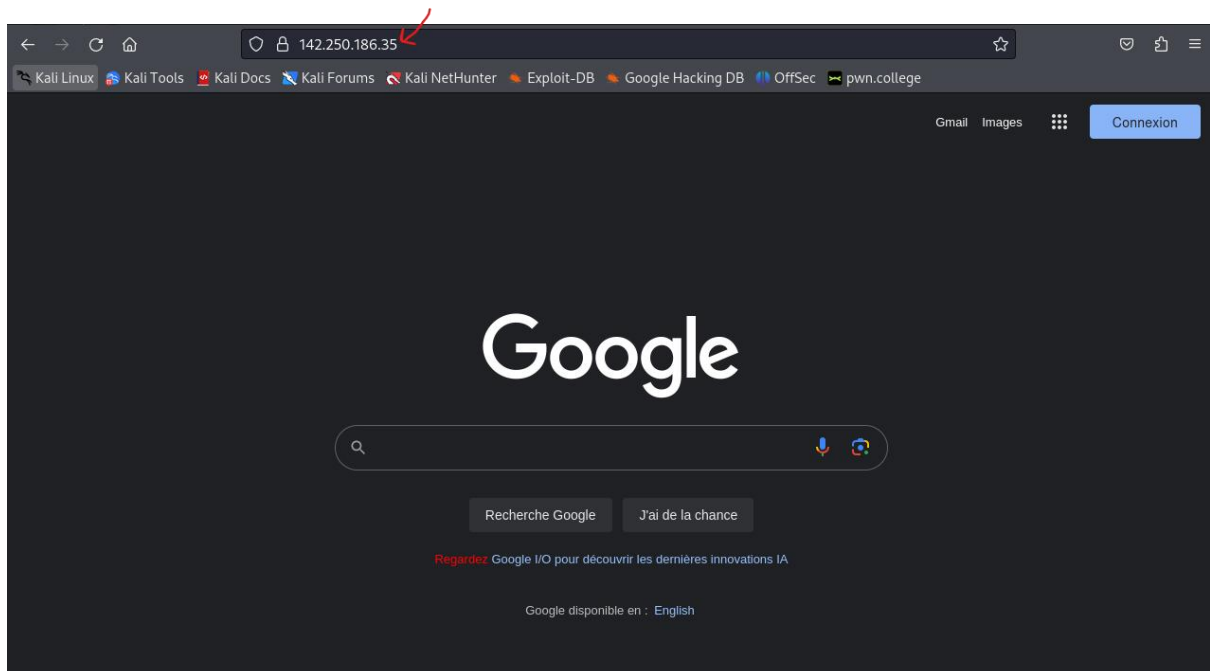
```
;; AUTHORITY SECTION:
google.fr. 3600 IN      NS      ns1.google.com.
google.fr. 3600 IN      NS      ns2.google.com.
google.fr. 3600 IN      NS      ns3.google.com.
google.fr. 3600 IN      NS      ns4.google.com.
```

Nous allons demander également à l'un de ces serveurs autoritaires de nous donner des infos sur le champ A du domaine `google.fr`

➤ *dig @ns1.google.com A google.fr*

```
paul@kaliLinux: ~  
File Actions Edit View Help  
(paul@kaliLinux)-[~]  
$ dig @ns1.google.com A google.fr  
  
;<<> DiG 9.19.19-1-Debian <<> @ns1.google.com A google.fr  
; (2 servers found)  
;; global options: +cmd  
;; Got answer:  
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 10583  
;; flags: qr aa rd; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1  
;; WARNING: recursion requested but not available  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags:; udp: 512  
;; QUESTION SECTION:  
;google.fr.                IN      A  
  
;; ANSWER SECTION:  
google.fr.                300     IN      A      142.250.186.35  
  
;; Query time: 160 msec  
;; SERVER: 216.239.32.10#53(ns1.google.com) (UDP)  
;; WHEN: Wed May 15 15:36:28 CEST 2024  
;; MSG SIZE rcvd: 54  
  
(paul@kaliLinux)-[~]  
$
```

Lorsque nous rentrons cette URL dans le navigateur, nous affichons bien la page correspondant à Google



3- Comprendre le fonctionnement de http

A Savoir : La différence entre un serveur et un serveur web réside dans leur fonction et leur utilisation.

Un serveur est un ordinateur ou un système informatique qui fournit des services, des ressources ou des fonctionnalités à d'autres ordinateurs, appelés clients, au sein d'un réseau. Il peut s'agir d'un serveur de fichiers, d'un serveur de messagerie, d'un serveur de bases de données, etc. En bref, un serveur est responsable de la gestion et de la distribution des ressources dans un réseau.

D'autre part, un serveur web est un type spécifique de serveur qui héberge des sites web et les rend accessibles sur Internet. Il répond aux requêtes des navigateurs web en envoyant les fichiers HTML, les

images, les scripts et autres ressources nécessaires pour afficher un site web sur un appareil client. Le serveur web utilise le protocole HTTP (Hypertext Transfer Protocol) pour communiquer avec les navigateurs et transférer les données demandées. En somme, le serveur web peut alors être considéré comme un service du serveur. Il communique en général sur le port 80 (port http standard)



En résumé, un serveur est un système informatique qui fournit des services dans un réseau, tandis qu'un serveur web est un type de serveur spécifiquement conçu pour héberger des sites web et les rendre accessibles sur Internet.

De quoi est constitué la connexion client/serveur qui est dite connexion http ?

Http est un protocole de connexion. Voici un exemple de requête



- Il s'agit ici d'une requête GET qui demande l'affichage d'une page web
- La page web en question est « / » qui représente la racine du domaine
- Http/2.0 est la version du protocole http utilisée
- L'url réel ressemble à ceci >> <http://www.leblogduhacker.fr/>
 - Wwww. = sous domaine
 - leblogduhacker.fr = domaine
 - / racine

Les requêtes http contiennent des en-têtes, (ici en bleu et valeur en violet). Ces en-têtes sont des informations que le navigateur va donner au serveur

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

Type accepté

Accept-Encoding: gzip, deflate, br

Encodage et langue accepté

Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3

Connection: keep-alive

Cookie: __cfduid=dcddc5190363f07d2642f...15010b0e772d68c6fe33da60b99

Le cookie (il y a donc déjà eu une connexion car c'est le serveur web qui place le cookie chez le client, qui lui le renvoie en cas de besoin)

DNT: 1

Host: www.leblogduhacker.fr

Host qui est l'hôte qu'on cherche à joindre

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 10.0; ...) Gecko/20100101 Firefox/58.0

L'agent Utilisateur qui contient les infos sur le navigateur et le système d'exploitation utilisé

Le serveur web répond à son tour



La réponse ici est http/2.0 200 OK

- Http/2.0 c'est la version du protocole http
- Code 200 ok, il s'agit d'un code de succès (commencent par 2...)
 - o Il existe différents types de code :

Code 1** informations

Code	Message	Signification
100	<i>Continue</i>	Attente de la suite de la requête.
101	<i>Switching Protocols</i>	Acceptation du changement de protocole.
102	<i>Processing</i>	WebDAV : Traitement en cours (évite que le client dépasse le temps d'attente limite).
103	<i>Early Hints</i>	RFC 8297 ³ : (Expérimental) Dans l'attente de la réponse définitive, le serveur retourne des liens que le client peut commencer à télécharger.

Code 3** Redirection

Code	Message	Signification
300	<i>Multiple Choices</i>	L'URI demandée se rapporte à plusieurs ressources.
301	<i>Moved Permanently</i>	Document déplacé de façon permanente.
302	<i>Found</i>	Document déplacé de façon temporaire.
303	<i>See Other</i>	La réponse à cette requête est ailleurs.
304	<i>Not Modified</i>	Document non modifié depuis la dernière requête.
305	<i>Use Proxy</i>	La requête doit être ré-adressée au proxy .
306	(aucun)	Code utilisé par une ancienne version de la RFC 2616 ⁵ , à présent réservé.
307	<i>Temporary Redirect</i>	La requête doit être redirigée temporairement vers l'URI spécifiée.
308	<i>Permanent Redirect</i>	La requête doit être redirigée définitivement vers l'URI spécifiée.
310	<i>Too many Redirects</i>	La requête doit être redirigée de trop nombreuses fois, ou est victime d'une boucle de redirection.

Code 4** Erreur coté client (réseau)

Code	Message	Signification
400	<i>Bad Request</i>	La syntaxe de la requête est erronée.
401	<i>Unauthorized</i>	Une authentification est nécessaire pour accéder à la ressource.
402	<i>Payment Required</i>	Paielement requis pour accéder à la ressource.
403	<i>Forbidden</i>	Le serveur a compris la requête, mais refuse de l'exécuter. Contrairement à l'erreur 401, s'authentifier ne fera aucune différence. Sur les serveurs où l'authentification est requise, cela signifie généralement que l'authentification a été acceptée mais que les droits d'accès ne permettent pas au client d'accéder à la ressource.
404	<i>Not Found</i>	Ressource non trouvée.
405	<i>Method Not Allowed</i>	Méthode de requête non autorisée.
[...]	[...]	[...]

Code 5** Erreur coté serveur web

Code	Message	Signification
500	<i>Internal Server Error</i>	Erreur interne du serveur.
501	<i>Not Implemented</i>	Fonctionnalité réclamée non supportée par le serveur.
502	<i>Bad Gateway ou Proxy Error</i>	Mauvaise réponse envoyée à un serveur intermédiaire par un autre serveur.
503	<i>Service Unavailable</i>	Service temporairement indisponible ou en maintenance.
504	<i>Gateway Time-out</i>	Temps d'attente d'une réponse d'un serveur à un serveur intermédiaire écoulé.
505	<i>HTTP Version not supported</i>	Version HTTP non gérée par le serveur.
506	<i>Variant Also Negotiates</i>	RFC 2295 ¹⁴ : Erreur de négociation. <i>Transparent content negociation</i> .
[...]	[...]	[...]

- Dans les en-têtes de réponse nous trouvons, le type du contenu, la date, le cookie (mis à jour) etc...

POST contact/ HTTP/2.0

```

Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip, deflate, br
Accept-Language: fr,fr-FR;q=0.8,en-US;q=0.5,en;q=0.3
Connection: keep-alive
Cookie: __cfduid=dcddc5190363f07d2642f...15010b0e772d68c6fe33da60b99
DNT: 1
Host: www.leblogduhacker.fr
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; ...) Gecko/20100101 Firefox/58.0
adressedecontact: admin@leblogduhacker.fr
message: Mon+message+de+contact
nom: Michel

```

Le navigateur peut également envoyer une requête avec la méthode POST

Nous constatons que les données sont quasiment les mêmes, à la différence qu'ici des paramètres ont été ajoutés ces informations

ont été définies par le web Master (via un formulaire en général) qui souhaite les communiquer au serveur

4- le protocole Https (chiffrement de communications)

Vient s'insérer dans la communication navigateur/serveur le « TLS » qui remplace le « SSL »

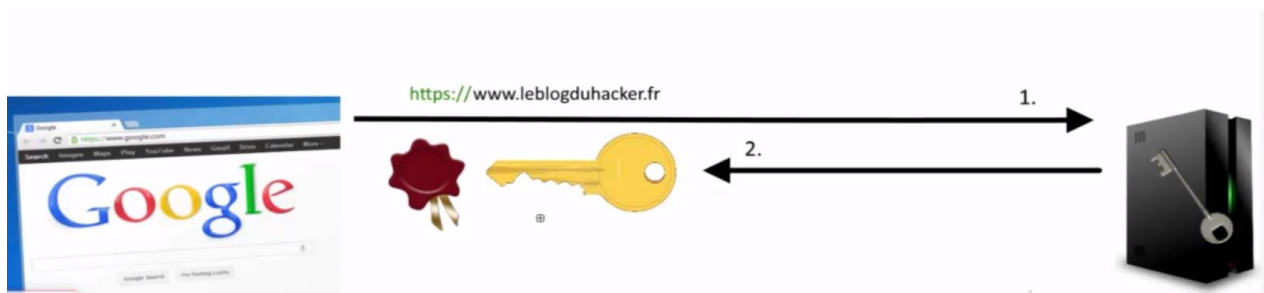
SSL (Secure Sockets Layer) et TLS (Transport Layer Security) sont des protocoles de sécurité utilisés pour sécuriser les communications sur Internet.

SSL était le protocole de sécurité original développé par Netscape Communications dans les années 1990. Il a été utilisé pour établir des connexions sécurisées entre un client (tel qu'un navigateur web) et un serveur, en garantissant la confidentialité, l'intégrité et l'authenticité des données échangées.

TLS est essentiellement une version améliorée et plus sécurisée de SSL. Il a été développé pour remplacer SSL et est désormais largement utilisé. TLS utilise des algorithmes de cryptage plus forts et dispose de mécanismes de sécurité améliorés pour se protéger contre les vulnérabilités et les attaques connues.

L'objectif principal de SSL et TLS est de sécuriser les communications en établissant une connexion chiffrée entre le client et le serveur. Cela permet de protéger les données sensibles telles que les informations d'identification, les informations personnelles et les transactions financières contre les interceptions et les manipulations par des tiers malveillants.

Pour pouvoir passer le TLS, le serveur possède deux clés : une clé secrète propre au serveur et une clé publique qui sera utilisée lors de la communication = Principe du chiffrement dit asymétrique

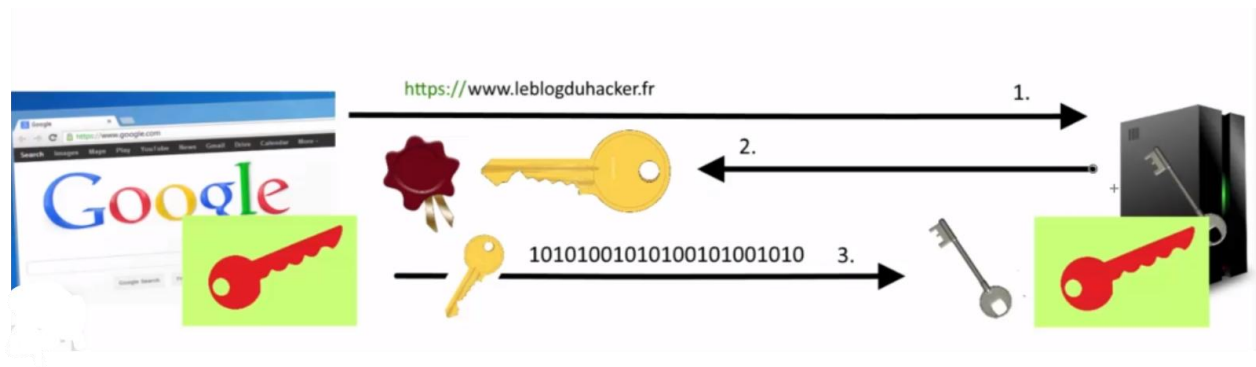


- La clé privée sert à déchiffrer des données chiffrées par la clé publique.

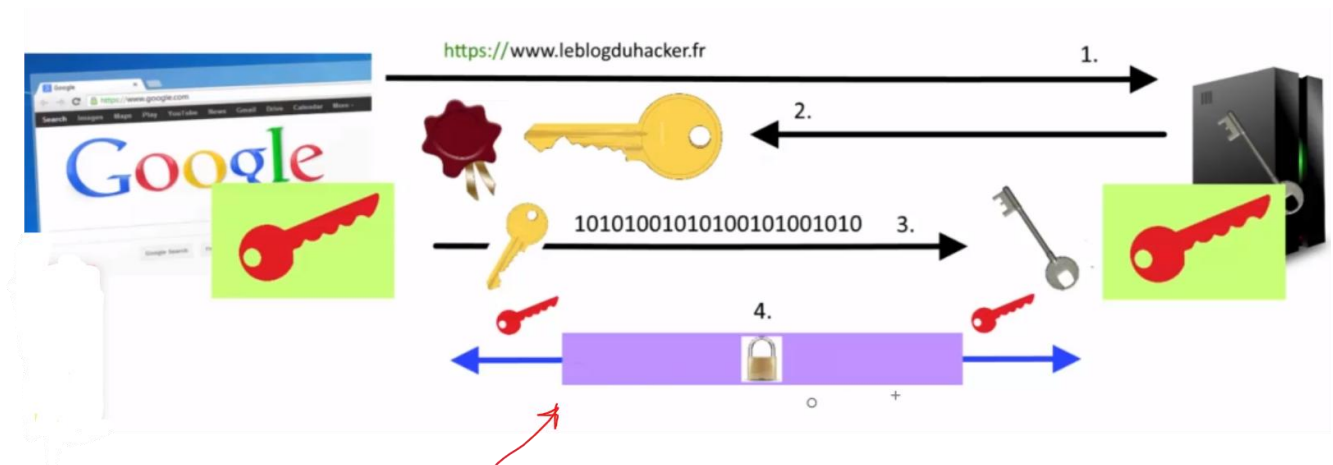
Ce processus nous garantit une authenticité car il serait impossible de déchiffrer un document à partir d'un autre serveur qui aurait une autre paire de clé.

- Le serveur web envoie également un certificat qui nous assure que la clé publique vient bien du bon site, pour ainsi vérifier son identité

Passé les politesses, le navigateur va générer une clé de session qui sera chiffrée via la clé public fournie par le serveur web. Lorsque cette clé de session chiffrée arrive au serveur en question, il pourra la déchiffrer à l'aide de sa clé privée et récupérer les données de session qui s'y trouve. Cette fois on parle de chiffrement symétrique car la clé de session est la même du navigateur au serveur



En dessous de ses vérifications par clés lors du protocole Https, nous avons nos requêtes classiques (GET/POST/PUT/DELETE)



Nos requêtes sont ainsi chiffrés, et toutes personnes qui souhaiterai voir leur contenues, devra avoir les clés permettant de déchiffrer les données